

ПРАКТИЧНЕ ЗАНЯТТЯ №16

Тема: Рефакторинг. Перевірка поведінки програми за допомогою Cucumber

Звіт про виконання практичного заняття

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Ознайомитися з поняттям рефакторингу, ознаками поганого коду, основними методами покращення структури програми та використанням Cucumber для перевірки поведінки програми після змін у коді.

Короткі теоретичні відомості

Рефакторинг - це процес покращення внутрішньої структури програмного коду без зміни зовнішньої поведінки програми. Його виконують для того, щоб код став простішим для читання, підтримки, тестування та подальшого розширення.

Під час рефакторингу важливо мати перевірки, які підтверджують, що поведінка програми не змінилася. Для цього можна використовувати Cucumber - інструмент, який дозволяє описувати сценарії поведінки у форматі Gherkin та перевіряти їх автоматично.

У цій роботі Cucumber використано для перевірки трьох сценаріїв: розрахунок ціни зі знижкою, вибір способу оплати та формування повідомлення після завершення замовлення.

Завдання

- дати відповіді на контрольні запитання з теми рефакторингу;
- створити приклад проекту з використанням Cucumber;
- описати поведінку програми у форматі Gherkin;
- реалізувати кроки перевірки для сценаріїв;
- запустити автоматичні перевірки та зафіксувати результат виконання.

Хід виконання роботи

Було створено проект Refactoring_Cucumber на базі Node.js. У проекті встановлено пакет @cucumber/cucumber, після чого створено структуру з папкою features та файлом кроків перевірки у step_definitions.

У файлі сценаріїв описано три поведінкові перевірки. Перша перевіряє правильність розрахунку кінцевої ціни зі знижкою. Друга перевіряє оброблення способу оплати. Третя перевіряє формування повідомлення після завершення замовлення.

Програмний код та файли сценаріїв розміщуються в репозиторії GitHub. У звіті наведено результат успішного запуску тестів.

Результат виконання

```
> cucumber-js  
  
.....  
  
3 scenarios (3 passed)  
9 steps (9 passed)  
0m00.011s (executing steps: 0m00.000s)  
PS C:\Users\Vadim\Desktop\Refactoring_Cucumber>
```

Рисунок 1 - Результат успішного запуску Cucumber-тестів

На рисунку видно, що всі сценарії та кроки виконано успішно: 3 scenarios passed, 9 steps passed. Це підтверджує, що описана поведінка програми відповідає очікуваному результату.

Контрольні завдання та запитання

1. Що таке рефакторинг?

Рефакторинг - це зміна внутрішньої структури коду без зміни зовнішньої поведінки програми. Після рефакторингу програма повинна працювати так само, як і до нього, але код має бути зрозумілішим, простішим для підтримки та зручнішим для подальшого розвитку.

2. Які ознаки поганого коду?

Ознаками поганого коду є дублювання, занадто довгі методи, великі класи з багатьма обов'язками, незрозумілі назви змінних і функцій, складні умови, надмірна кількість параметрів, сильна залежність між класами та наявність коду, який важко тестувати або змінювати.

3. Які ви знаєте методи рефакторингу?

До поширених методів рефакторингу належать виділення методу, перейменування змінних і функцій, виділення класу, переміщення методу в інший клас, інкапсуляція поля, видалення дублювання, спрощення умовних операторів та заміна умовного оператора поліморфізмом.

4. Коли треба замінювати оператор switch поліморфізмом?

Оператор switch варто замінювати поліморфізмом тоді, коли він використовується для вибору поведінки залежно від типу об'єкта і постійно розширюється новими варіантами. Якщо однаковий switch повторюється в різних частинах програми, краще створити базовий клас або інтерфейс і реалізувати різну поведінку в окремих похідних класах.

5. Яким чином та як часто треба виконувати рефакторинг?

Рефакторинг потрібно виконувати регулярно, невеликими кроками. Найкраще робити його під час роботи з кодом: перед додаванням нової функціональності, після виявлення дублювання, при спрощенні складного методу або після виправлення помилки. Після кожної зміни бажано запускати тести, щоб переконатися, що поведінка програми не змінилася.

Висновок

У ході виконання практичного заняття було розглянуто поняття рефакторингу, основні ознаки поганого коду та методи покращення структури програми. Також було створено приклад проекту з використанням Cucumber для автоматичної перевірки поведінки програми.

Практична частина показала, що сценарії у форматі Gherkin дозволяють описувати очікувану поведінку зрозумілою мовою, а автоматичний запуск Cucumber-тестів допомагає перевірити, що після змін у коді програма продовжує працювати правильно. Це робить рефакторинг безпечнішим і зручнішим.